

Machine Learning

Dr. Indu Joshi

Assistant Professor at
Indian Institute of Technology Mandi

18 March 2025

Support Vector Machine

- The Support Vector Machine (SVM) finds the maximum margin separating hyperplane.
- We define a linear classifier:

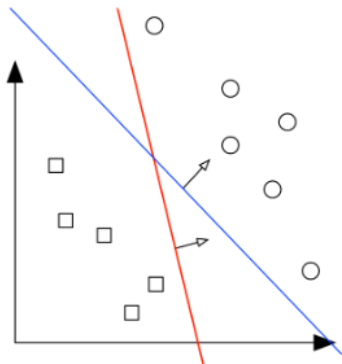
$$h(x) = \text{sign}(\mathbf{w}^T \mathbf{x} + b)$$

and we assume a binary classification setting with labels $\{+1, -1\}$.

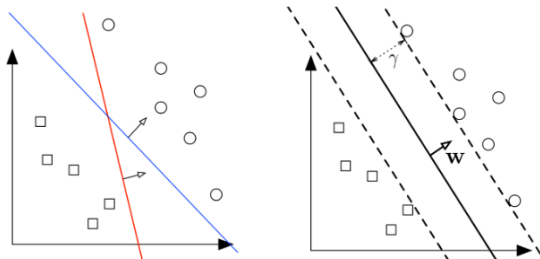
Question

- Typically, if a data set is linearly separable, there are infinitely many separating hyperplanes. A natural question to ask is:
- What is the best separating hyperplane?

Which is a better classifier?



Better Classifier



- (Left:) Two different separating hyperplanes for the same data set.
- (Right:) The maximum margin hyperplane. The margin, γ , is the distance from the hyperplane (solid line) to the closest points in either class (which touch the parallel dotted lines).

SVM Answer

- The one that maximizes the distance to the closest data points from both classes. We say it is the hyperplane with maximum margin.

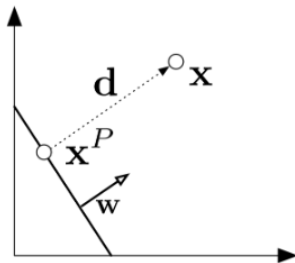
Margin

- A hyperplane is defined through $\mathbf{w}$, b as a set of points such that:

$$H = \{\mathbf{x} \mid \mathbf{w}^T \mathbf{x} + b = 0\}.$$

- Let the margin γ be defined as the distance from the hyperplane to the closest point across both classes.
- What is the distance of a point $\mathbf{x}$ to the hyperplane H ?

Margin



- Consider some point x . Let d be the vector from H to x of minimum length.
- Let x_P be the projection of x onto H . It follows then that:

$$x_P = x - d.$$

- Since d is parallel to w , we can write:

$$d = \alpha w, \quad \text{for some } \alpha \in \mathbb{R}.$$

Margin

- As $\mathbf{x}_P \in H$, it satisfies:

$$\mathbf{w}^T \mathbf{x}_P + b = 0.$$

- Substituting $\mathbf{x}_P = \mathbf{x} - \mathbf{d}$:

$$\mathbf{w}^T (\mathbf{x} - \alpha \mathbf{w}) + b = 0.$$

$$\implies \alpha = \frac{\mathbf{w}^T \mathbf{x} + b}{\mathbf{w}^T \mathbf{w}}.$$

The length of $\mathbf{d}$ is given by:

$$\begin{aligned} \|\mathbf{d}\|_2 &= \sqrt{\mathbf{d}^T \mathbf{d}} = \sqrt{\alpha^2 \mathbf{w}^T \mathbf{w}} = \alpha \|\mathbf{w}\|_2 = \frac{|\mathbf{w}^T \mathbf{x} + b| \|\mathbf{w}\|_2}{\|\mathbf{w}\|_2^2} \\ &= \frac{|\mathbf{w}^T \mathbf{x} + b|}{\|\mathbf{w}\|_2}. \end{aligned}$$

Margin

- The margin of H with respect to D is:

$$\gamma(\mathbf{w}, b) = \min_{\mathbf{x} \in D} \frac{|\mathbf{w}^T \mathbf{x} + b|}{\|\mathbf{w}\|_2}.$$

- By definition, the margin and hyperplane are scale invariant:

$$\gamma(\beta \mathbf{w}, \beta b) = \gamma(\mathbf{w}, b), \quad \forall \beta \neq 0.$$

- Note that if the hyperplane is such that γ is maximized, it must lie right in the middle of the two classes.
- In other words, γ must be the distance to the closest point within both classes.
- If not, one could move the hyperplane towards data points of the class that is further away and increase γ , which contradicts that γ is maximized.

Maximum Margin Classifier

- We can formulate our search for the maximum margin separating hyperplane as a constrained optimization problem.
- Let us assume that the training data set is linearly separable in feature space, so that by definition there exists at least one choice of the parameters w and b , such that $w^T(x_i) + b > 0$ for points having $y_i = +1$ and $w^T(x_i) + b < 0$ for points having $y_i = -1$, so that $y_i(w^T(x_i) + b) > 0$ for all training data points.

Maximum Margin Classifier

- The objective is to maximize the margin under the constraints that all data points must lie on the correct side of the hyperplane:

$$\max_{\mathbf{w}, b} \underbrace{\gamma(\mathbf{w}, b)}_{\text{maximize margin}} \quad \text{such that} \quad \underbrace{\forall i \quad y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 0}_{\text{separating hyperplane}}$$

Maximum Margin Classifier

- If we plug in the definition of γ we obtain:

$$\max_{\mathbf{w}, b} \underbrace{\frac{1}{\|\mathbf{w}\|_2} \min_{x_i \in D} |\mathbf{w}^T \mathbf{x}_i + b|}_{\gamma(\mathbf{w}, b) \text{ (maximize margin)}} \quad \text{s.t.} \quad \underbrace{\forall i \quad y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 0}_{\text{separating hyperplane}}$$

- Because the hyperplane is scale-invariant, we can fix the scale of w, b any way we want.
- Let's be clever about it and choose it such that:

$$\min_{x \in D} |w^T x + b| = 1.$$

Why the hyperplane is scale-invariant?

A hyperplane is *scale-invariant* because if we multiply both $\mathbf{w}$ and b by a nonzero scalar λ , the set of points satisfying the equation remains unchanged.

The equation of the hyperplane is given by:

$$\mathbf{w}^T \mathbf{x} + b = 0.$$

Now, if we scale $\mathbf{w}$ and b by some nonzero scalar λ , we get:

$$(\lambda \mathbf{w})^T \mathbf{x} + \lambda b = 0.$$

Rewriting, we get:

$$\lambda(\mathbf{w}^T \mathbf{x} + b) = 0.$$

Since $\lambda \neq 0$, this equation is equivalent to the original equation $\mathbf{w}^T \mathbf{x} + b = 0$.

How can we make $\min_{\mathbf{x} \in D} |\mathbf{w}^T \mathbf{x} + b| = 1$?

Since the hyperplane equation

$$\mathbf{w}^T \mathbf{x} + b = 0$$

is scale-invariant, we can multiply $\mathbf{w}$ and b by a suitable scalar without changing the set of points on the hyperplane. This allows us to enforce the desired condition. To achieve

$$\min_{\mathbf{x} \in D} |\mathbf{w}^T \mathbf{x} + b| = 1,$$

define the minimum margin value:

$$m = \min_{\mathbf{x} \in D} |\mathbf{w}^T \mathbf{x} + b|.$$

If $m \neq 1$, we rescale $\mathbf{w}$ and b by dividing them by m :

$$\mathbf{w}' = \frac{\mathbf{w}}{m}, \quad b' = \frac{b}{m}.$$

How can we make $\min_{x \in D} |w^T x + b| = 1$?

Substituting these into the hyperplane equation:

$$\mathbf{w}'^T \mathbf{x} + b' = \frac{1}{m}(\mathbf{w}^T \mathbf{x} + b).$$

Thus, the minimum value of $|\mathbf{w}'^T \mathbf{x} + b'|$ over D becomes:

$$\frac{1}{m} \min_{\mathbf{x} \in D} |\mathbf{w}^T \mathbf{x} + b| = \frac{1}{m} m = 1.$$

This ensures that the minimum distance from any point in D to the hyperplane is exactly 1.

Maximum Margin Classifier

- We can add this re-scaling as an equality constraint. Then our objective becomes:

$$\max_{w,b} \frac{1}{\|w\|_2} \cdot 1 = \min_{w,b} \|w\|_2^2$$

(Where we made use of the fact that $f(z) = z^2$ is a monotonically increasing function for $z \geq 0$ and $\|w\| \geq 0$; i.e., the w that maximizes $\|w\|_2$ also maximizes $w^T w$.)

The new optimization problem becomes:

$$\min_{w,b} \frac{1}{2} \|w\|_2^2 \quad \text{s.t.} \quad \forall i, \quad y_i(w^T x_i + b) \geq 0, \min_i |w^T x_i + b| = 1.$$

Maximum Margin Classifier

- The optimization problem becomes:

$$\min_{w,b} \frac{1}{2} \|w\|_2^2 \quad \text{s.t.} \quad \forall i, \quad y_i(w^T x_i + b) \geq 1.$$

- The bias parameter b is determined implicitly via the constraints, because these require that changes to w be compensated by changes to b .
- It has a unique solution whenever a separating hyperplane exists.
- It also has a nice interpretation: Find the simplest hyperplane (where simpler means smaller $\|w\|_2^2$) such that all inputs lie at least 1 unit away from the hyperplane on the correct side.

Constraint

$$y_i(w^T x_i + b) \geq 1$$

$$\implies y_i(w^T x_i + b) - 1 \geq 0$$

$$\implies -(y_i(w^T x_i + b) - 1) \leq 0$$

Constrained Optimization Problem

The general form of a constrained optimization problem is given as:

$$(P2) \min f(x)$$

$$\text{s.t. } g_i(x) \leq 0; i = 1, 2, \dots, m$$

where f and $g_i(\forall i) : \mathbb{R}^n \rightarrow \mathbb{R}$ are defined and continuously differentiable functions.

KKT Conditions: Necessary and Sufficient for Convex Optimization

Let $\bar{x}$ be the global minimum point of the problem at which basic constraint qualification holds. Then there exist multipliers (KKT multipliers) $\bar{\lambda}_i, i = 1, 2, \dots, m$ such that the following conditions hold:

1. $\nabla f(\bar{x}) + \sum_{i=1}^m \bar{\lambda}_i \nabla g_i(\bar{x}) = 0$
2. $g_i(\bar{x}) \leq 0; i = 1, 2, \dots, m$
3. $\bar{\lambda}_i g_i(\bar{x}) = 0; i = 1, 2, \dots, m$
4. $\bar{\lambda}_i \geq 0$ for all i

These conditions are called KKT Conditions.

Constrained Optimization: KKT Conditions

- We introduce Lagrange multipliers $\alpha_i \geq 0$ and construct the Lagrangian:

$$L(w, b, \alpha) = \frac{1}{2} \|w\|_2^2 - \sum_{i=1}^n \alpha_i (y_i (w^T x_i + b) - 1).$$

Taking derivatives:

- With respect to w :

$$\frac{\partial L}{\partial w} = w - \sum_{i=1}^n \alpha_i y_i x_i = 0 \Rightarrow w = \sum_{i=1}^n \alpha_i y_i x_i.$$

Constrained Optimization: KKT Conditions

- With respect to b :

$$\frac{\partial L}{\partial b} = - \sum_{i=1}^n \alpha_i y_i = 0.$$

Thus, we obtain the constraint $\sum_{i=1}^n \alpha_i y_i = 0$.

- $\alpha_i \geq 0 \quad \forall i$
- $y_i(w^T x_i + b) - 1 \geq 0$
- $\alpha_i(y_i(w^T x_i + b) - 1) = 0$

Take Away

- Thus for every data point, either $\alpha_i = 0$ or $y_i(w^T x_i + b) = 1$.
- Any data point for which $\alpha_i = 0$ will not appear in the summation for the prediction $(h(x) = w^T x + b, w = \sum_{i=1}^n \alpha_i y_i x_i)$ and hence plays no role in making predictions for new data points.
- The remaining training data points are called *support vectors*, and because they satisfy $y_i(w^T x_i + b) = 1$, they correspond to points that lie on the maximum margin hyperplanes in feature space.
- This property is central to the practical applicability of support vector machines. Once the model is trained, a significant proportion of the data points can be discarded and only the support vectors retained.

Support Vectors

- Support vectors are special because they are the training points that define the maximum margin of the hyperplane to the data set, and they therefore determine the shape of the hyperplane.
- If you were to move one of them and retrain the SVM, the resulting hyperplane would change.
- The opposite is the case for non-support vectors (provided you don't move them too much, or they would turn into support vectors themselves).
- This will become particularly important in the dual formulation for Kernel-SVMs.

Dual Formulation

- Substituting $w = \sum_i \alpha_i y_i x_i$ into the objective function:

$$\frac{1}{2} \|w\|_2^2 = \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i^T x_j.$$

The Lagrangian is given by:

$$L(\alpha) = \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i^T x_j - \sum_{i=1}^n \alpha_i y_i (w^T x_i + b) + \sum_{i=1}^n \alpha_i.$$

$$\sum_{i=1}^n \alpha_i y_i w^T x_i = \sum_{i=1}^n \alpha_i y_i \left(\sum_{j=1}^n \alpha_j y_j x_j^T x_i \right),$$

$$\Rightarrow L(\alpha) = \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i^T x_j - \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i^T x_j + \sum_{i=1}^n \alpha_i$$

Dual Formulation

$$\Rightarrow L(\alpha) = -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i^T x_j + \sum_{i=1}^n \alpha_i$$

subject to:

$$\sum_{i=1}^n \alpha_i y_i = 0, \quad \alpha_i \geq 0, \quad \forall i.$$

- Thus, the dual problem is:

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j x_i^T x_j,$$

subject to:

$$\sum_i \alpha_i y_i = 0, \quad \alpha_i \geq 0, \quad \forall i$$

Optimal Solution

- Values of α_i are obtained using quadratic programming methods.
- Once α_i values are obtained: $w = \sum_i \alpha_i y_i x_i$.
- The bias term b is computed from any support vector satisfying $y_i(w^T x_i + b) = 1$.
- A numerically more stable solution is obtained by first multiplying through by y_n , making use of $y_n^2 = 1$, and then averaging these equations over all support vectors and solving for b to give:

$$b = y_i - w^T x_i$$

Substituting $w = \sum_j \alpha_j y_j x_j$, we get:

$$b = y_i - \sum_j \alpha_j y_j x_j^T x_i$$

Optimal Solution

- In practice, different support vectors might give slightly different b values due to rounding or noise. So, instead of using just one support vector, we average over all of them for stability.

$$b = \frac{1}{N_S} \sum_{i \in S} \left(y_i - \sum_{j \in S} \alpha_j y_j x_i^T x_j \right)$$

- S denotes the set of indices of the support vectors. Thus, the optimal hyperplane equation is:

$$w^T x + b = 0.$$

SVM with soft constraints

- If the data is low-dimensional, it is often the case that there is no separating hyperplane between the two classes (data not linearly separable).
- This means that a hard-margin SVM, which strictly requires a perfect separation of classes, fails to find a solution.
- In this case, there is no solution to the optimization problems stated above.
- We can fix this by allowing the constraints to be violated ever so slightly with the introduction of slack variables:

$$\begin{aligned} \min_{w,b} \quad & w^T w + C \sum_{i=1}^n \xi_i \\ \text{s.t.} \quad & \forall i, \quad y_i(w^T x_i + b) \geq 1 - \xi_i \\ & \forall i, \quad \xi_i \geq 0 \end{aligned}$$

Soft-Margin SVM and Allowing Misclassification

- **Correctly Classified and Outside the Margin** ($\xi_i = 0$):

$$y_i(w^T x_i + b) \geq 1$$

The point is correctly classified and outside (or on) the margin, similar to hard-margin SVM.

- **Correctly Classified but Inside the Margin** ($0 < \xi_i \leq 1$):

$$0 \leq y_i(w^T x_i + b) < 1$$

The point is correctly classified but lies within the margin.

- **Misclassified** ($\xi_i > 1$):

$$y_i(w^T x_i + b) < 0$$

The point is misclassified and falls on the wrong side of the decision boundary.

SVM with soft constraints

- The slack variable ξ_i allows the input x_i to be closer to the hyperplane (or even be on the wrong side), but there is a penalty in the objective function for such “slack”.
- If C is very large, the SVM becomes very strict and tries to get all points to be on the right side of the hyperplane.
- If C is very small, the SVM becomes very loose and may “sacrifice” some points to obtain a simpler (i.e., lower $\|w\|_2^2$) solution.

Unconstrained Formulation

- Let us consider the value of ξ_i for the case of $C \neq 0$.
- Because the objective will always try to minimize ξ_i as much as possible, the equation must hold as an equality and we have:

$$\xi_i = \begin{cases} 1 - y_i(\mathbf{w}^T \mathbf{x}_i + b), & \text{if } y_i(\mathbf{w}^T \mathbf{x}_i + b) < 1, \\ 0, & \text{if } y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1. \end{cases}$$

This is equivalent to the following closed form:

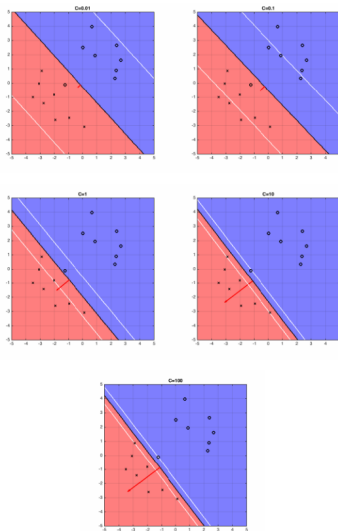
$$\xi_i = \max(1 - y_i(\mathbf{w}^T \mathbf{x}_i + b), 0).$$

Unconstrained Formulation

- If we plug this closed form into the objective of our SVM optimization problem, we obtain the following unconstrained version as loss function and regularizer:

$$\min_{\mathbf{w}, b} \underbrace{\mathbf{w}^T \mathbf{w}}_{\ell_2\text{-regularizer}} + C \sum_{i=1}^n \underbrace{\max [1 - y_i(\mathbf{w}^T \mathbf{x}_i + b), 0]}_{\text{hinge-loss}}$$

Hyperplane

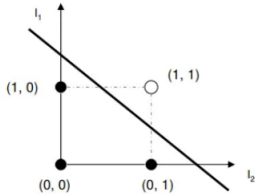


Cover's Theorem

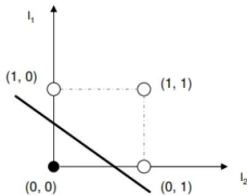
- A complex pattern classification problem that is not linearly separable in a low-dimensional space is more likely to become linearly separable when mapped to a higher-dimensional space using a non-linear transformation.
- If your data is not linearly separable in a lower-dimensional space, projecting it into a higher-dimensional space (possibly infinite) increases the likelihood of finding a linear separation.
- This is the theoretical basis for the Kernel Trick in SVMs.

Example- XOR Function

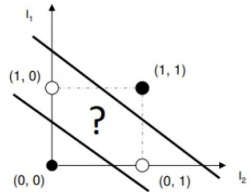
AND		
I_1	I_2	out
0	0	0
0	1	0
1	0	0
1	1	1



OR		
I_1	I_2	out
0	0	0
0	1	1
1	0	1
1	1	1

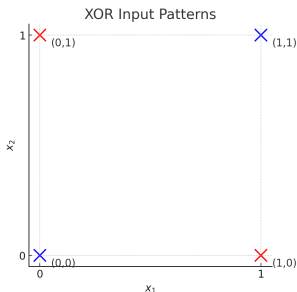


XOR		
I_1	I_2	out
0	0	0
0	1	1
1	0	1
1	1	0



XOR Gate: Non-linear Separability

- The requirement is to construct a pattern classifier that outputs binary 0 in response to the input patterns $(1, 1)$ or $(0, 0)$, and outputs binary 1 in response to the input patterns $(0, 1)$ or $(1, 0)$.



XOR Gate: Transformed Inputs

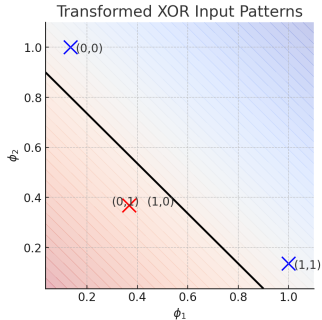
- Without even increasing the dimensionality of the input space, merely transforming the inputs using Gaussian functions is sufficient to transform the XOR problem into a linearly separable one.

$$\phi_1(x) = e^{-\|x-t_1\|^2}, \quad t_1 = [1, 1]^T$$

$$\phi_2(x) = e^{-\|x-t_2\|^2}, \quad t_2 = [0, 0]^T$$

Input Pattern x	$\phi_1(x)$	$\phi_2(x)$
(1,1)	1.0000	0.1353
(0,1)	0.3678	0.3678
(0,0)	0.1353	1.0000
(1,0)	0.3678	0.3678

XOR Gate: Transformed Inputs



Transformed inputs have become linearly separable.

Handcrafted Feature Expansion

- We can make linear classifiers non-linear by applying a **basis function** (feature transformation) on the input feature vectors.
- Formally, for a data vector $x \in \mathbb{R}^d$, we apply the transformation:

$$x \rightarrow \phi(x)$$

where $\phi(x) \in \mathbb{R}^D$. Usually, $D \gg d$ because we add dimensions that capture non-linear interactions among the original features.

Handcrafted Feature Expansion: Advantages and Disadvantages

Advantages:

- It is simple and keeps the optimization problem **convex** and well-behaved.

Disadvantages:

- The transformed feature vector $\phi(x)$ might be **very high dimensional**, leading to increased computational cost.

Example: Feature Transformation

Consider a feature vector:

$$x = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{pmatrix}$$

We define the feature transformation $\phi(x)$ such that it expands the feature space to include interaction terms between features, allowing a linear model to learn more complex decision boundaries.

Example: Feature Transformation

$$\phi(x) = \begin{pmatrix} 1 \\ x_1 \\ \vdots \\ x_d \\ x_1 x_2 \\ \vdots \\ x_{d-1} x_d \\ \vdots \\ x_1 x_2 \cdots x_d \end{pmatrix}$$

Inner-Product Computation Using Kernel

- The kernel trick allows us to compute the inner product in a high-dimensional feature space without explicitly transforming the data into that space.
- Instead of computing this transformation explicitly and then performing an inner product in $\mathbb{R}^D$, we use a **kernel function** that directly computes the inner product:

$$K(x, y) = \langle \phi(x), \phi(y) \rangle$$

- This means that rather than working with the transformed feature vectors $\phi(x)$ and $\phi(y)$ explicitly, we can compute their dot product using $K(x, y)$, which is typically much faster.

Inner-Product Computation Using Kernel: Example

Consider a feature transformation:

$$\phi(x) = (1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, x_2^2, \sqrt{2}x_1x_2)$$

Computing the dot product in this transformed space takes $O(D) = O(6)$ operations.

However, using the polynomial kernel:

$$K(x, y) = (x^T y + c)^2$$

we achieve the same result without explicitly computing $\phi(x)$ and $\phi(y)$ in $O(d) = O(2)$ operations, avoiding explicit transformation.

Inner-Product Computation Using Kernel: Example

$$K(x, y) = (x^T y + 1)^2 = (x_1 y_1 + x_2 y_2 + 1)^2$$

$$= x_1^2 y_1^2 + x_2^2 y_2^2 + 2x_1 x_2 y_1 y_2 + 2x_1 y_1 + 2x_2 y_2 + 1$$

$$\phi(x) = (1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, x_2^2, \sqrt{2}x_1 x_2)$$

$$\phi(y) = (1, \sqrt{2}y_1, \sqrt{2}y_2, y_1^2, y_2^2, \sqrt{2}y_1 y_2)$$

$$\begin{aligned}\phi(x)^T \phi(y) &= 1 \cdot 1 + (\sqrt{2}x_1)(\sqrt{2}y_1) + (\sqrt{2}x_2)(\sqrt{2}y_2) \\ &\quad + x_1^2 y_1^2 + x_2^2 y_2^2 + (\sqrt{2}x_1 x_2)(\sqrt{2}y_1 y_2) \\ &= 1 + 2x_1 y_1 + 2x_2 y_2 + x_1^2 y_1^2 + x_2^2 y_2^2 + 2x_1 x_2 y_1 y_2 \\ &\implies K(x, y) = \phi(x)^T \phi(y)\end{aligned}$$

Kernel Functions

- **Linear Kernel:** The linear kernel is equivalent to using a standard linear classifier. However, using a kernel matrix can be computationally efficient if the data dimensionality d is very high.

$$K(x, z) = x^{\top} z$$

- **Polynomial Kernel:**

$$K(x, z) = (1 + x^{\top} z)^d$$

Kernel Functions

- **Radial Basis Function (RBF) Kernel (Gaussian Kernel):**

$$K(x, z) = e^{-\frac{\|x-z\|^2}{\sigma^2}}$$

The RBF kernel is the most widely used kernel. It is a **universal approximator**, meaning it can approximate any continuous function given enough data. The corresponding feature mapping is **infinite-dimensional** and cannot be computed explicitly. However, efficient low-dimensional approximations exist.

Can any Function be a Kernel?

- No, not every function $K(x, y)$ can serve as a valid kernel function.
- The matrix $K(x_i, x_j)$ must correspond to real inner products after some transformation $x \rightarrow \phi(x)$.
- This holds **if and only if** K is **positive semi-definite (p.s.d.)** (Mercer's Theorem)
- A matrix $A \in \mathbb{R}^{n \times n}$ is **positive semi-definite** if and only if:

$$\forall q \in \mathbb{R}^n, \quad q^\top A q \geq 0$$

Can any Function be a Kernel?

- Since the kernel function represents an inner product in a transformed space, we define:

$$K_{ij} = \phi(x_i)^\top \phi(x_j)$$

- Thus, the kernel matrix K can be written as:

$$K = \phi^\top \phi$$

where:

$$\phi = [\phi(x_1), \dots, \phi(x_n)]$$

- follows that K is positive semi-definite because:

Can any Function be a Kernel?

$$\begin{aligned} q^\top K q &= q^\top (\phi^\top \phi) q \\ &= (\phi q)^\top (\phi q) \\ &= \|\phi q\|^2 \\ &\geq 0 \end{aligned}$$

Can any Function be a Kernel?

- Conversely, if any matrix A is p.s.d., it can be decomposed as:

$$A = \phi^\top \phi$$

for some realization of ϕ .

- This confirms that valid kernel functions must always generate p.s.d. kernel matrices.

Well-defined Kernels

- Kernels built by recursively combining one or more of the following rules are called **well-defined kernels**:

$$k(x, z) = x^\top z$$

$$k(x, z) = ck_1(x, z), \quad c \geq 0$$

$$k(x, z) = k_1(x, z) + k_2(x, z)$$

$$k(x, z) = g(k_1(x, z))$$

$$k(x, z) = k_1(x, z)k_2(x, z)$$

$$k(x, z) = f(x)k_1(x, z)f(z)$$

$$k(x, z) = e^{k_1(x, z)}$$

$$k(x, z) = x^\top Az, \quad A \succeq 0$$

Well-defined Kernels

- where:
 - k_1, k_2 are well-defined kernels,
 - $c \geq 0$ is a non-negative constant,
 - g is a polynomial function with positive coefficients,
 - f is any function,
 - $A \succeq 0$ is a positive semi-definite matrix.

Well-defined Kernels

- A kernel being well-defined is equivalent to the corresponding kernel matrix K being **positive semi-definite (p.s.d.)**, though we do not prove it here. This is equivalent to any of the following statements:
 - All eigenvalues of K are non-negative.
 - There exists a real matrix P such that:

$$K = P^{\top} P.$$

- For every real vector x , we have:

$$x^{\top} K x \geq 0.$$

Prove That the RBF Kernel is Well-Defined!

We can derive the RBF kernel using well-defined kernel rules:

$$k_1(x, z) = x^\top z \quad (\text{Rule 1})$$

$$k_2(x, z) = \frac{2}{\sigma^2} k_1(x, z) = \frac{2}{\sigma^2} x^\top z \quad (\text{Rule 2})$$

$$k_3(x, z) = e^{k_2(x, z)} = e^{\frac{2x^\top z}{\sigma^2}} \quad (\text{Rule 7})$$

$$k_4(x, z) = e^{-\frac{x^\top x}{\sigma^2}} k_3(x, z) e^{-\frac{z^\top z}{\sigma^2}}$$

$$(\text{Rule 6 with } f(x) = e^{-\frac{x^\top x}{\sigma^2}})$$

Prove That the RBF Kernel is Well-Defined!

$$\begin{aligned} &= e^{-\frac{x^T x}{\sigma^2}} e^{\frac{2x^T z}{\sigma^2}} e^{-\frac{z^T z}{\sigma^2}} \\ &= e^{-\frac{(x^T x - 2x^T z + z^T z)}{\sigma^2}} \\ &= e^{-\frac{\|x-z\|^2}{\sigma^2}} \\ &= k_{\text{RBF}}(x, z) \end{aligned}$$

Thus, the RBF kernel is a well-defined kernel.

Kernel SVM

- The original, primal Support Vector Machine (SVM) is a quadratic programming problem:

$$\begin{aligned} \min_{w,b} \quad & \frac{1}{2} w^\top w + C \sum_{i=1}^n \xi_i \\ \text{s.t.} \quad & y_i(w^\top x_i + b) \geq 1 - \xi_i, \quad \forall i \\ & \xi_i \geq 0 \end{aligned}$$

- The problem can be rewritten in its dual form:

$$\begin{aligned} \min_{\alpha_1, \dots, \alpha_n} \quad & \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j K_{ij} - \sum_{i=1}^n \alpha_i \\ \text{s.t.} \quad & 0 \leq \alpha_i \leq C, \quad \forall i \\ & \sum_{i=1}^n \alpha_i y_i = 0 \end{aligned}$$

Kernel SVM

where the weight vector w is given by:

$$w = \sum_{i=1}^n \alpha_i y_i \phi(x_i)$$

$$h(x) = \text{sign} \left(\sum_{i=1}^n \alpha_i y_i k(x_i, x) + b \right).$$

Kernel SVM

We begin with the primal form of the soft-margin Support Vector Machine (SVM):

$$\begin{aligned} \min_{w,b,\xi} \quad & \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \\ \text{s.t.} \quad & y_i(w^\top x_i + b) \geq 1 - \xi_i, \quad \forall i \\ & \xi_i \geq 0 \end{aligned}$$

The Lagrangian is:

$$\begin{aligned} \mathcal{L}(w, b, \xi, \alpha, \mu) = & \frac{1}{2} \|w\|^2 + C \sum_i \xi_i - \sum_i \alpha_i \left[y_i(w^\top x_i + b) - 1 + \xi_i \right] \\ & - \sum_i \mu_i \xi_i \end{aligned}$$

Kernel SVM

Take derivatives and apply KKT conditions:

$$\frac{\partial \mathcal{L}}{\partial w} = 0 \Rightarrow w = \sum_i \alpha_i y_i x_i$$

$$\frac{\partial \mathcal{L}}{\partial b} = 0 \Rightarrow \sum_i \alpha_i y_i = 0$$

$$\frac{\partial \mathcal{L}}{\partial \xi_i} = 0 \Rightarrow \alpha_i + \mu_i = C \Rightarrow 0 \leq \alpha_i \leq C$$

Substitute into the Lagrangian, yielding the dual problem:

$$\begin{aligned} \min_{\alpha} \quad & \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j x_i^\top x_j - \sum_i \alpha_i \\ \text{s.t.} \quad & 0 \leq \alpha_i \leq C, \quad \forall i \\ & \sum_i \alpha_i y_i = 0 \end{aligned}$$

Support Vectors

- From the primal formulation, we know that only support vectors satisfy the constraint with equality:

$$y_i(w^\top \phi(x_i) + b) = 1.$$

- In the dual formulation, the same training inputs can be identified by their corresponding dual values, where only those with $\alpha_i > 0$ are support vectors.
- All other training inputs have $\alpha_i = 0$ and do not contribute to the final model.
- At test time, we only need to compute the sum in $h(x)$ over the support vectors. Inputs x_i with $\alpha_i = 0$ can be discarded after training.

Calculating b

- One apparent issue with the dual formulation is that b is no longer part of the optimization problem. However, we need b to perform classification.
- Since the primal and dual solutions are identical, we can recover b using the support vectors, which satisfy:

$$y_i(w^\top \phi(x_i) + b) = 1.$$

$$y_i \left(\sum_j y_j \alpha_j k(x_j, x_i) + b \right) = 1.$$

$$b = y_i - \sum_j y_j \alpha_j k(x_j, x_i).$$

- In practice, it is best to average b over several support vectors to reduce numerical precision issues.

Thank You

Contact: indujoshi@iitmandi.ac.in